LEXICAL ANALYSIS – TOKEN DENSITY INDICATOR (C IMPLEMENTATION)

NAME: DEEKSHITA SARAVANAN

REG NO.: RA2311026050031

BRANCH: CSE-AIML ‘A’

COURSE :COMPILER DESIGN

Abstract

Lexical analysis is the first phase of a compiler, responsible for converting source code into tokens. However, raw token streams do not provide meaningful insights into code complexity or maintainability. This project introduces a metric called **Token Density**, defined as the number of tokens per line of code. The objective is to transform lexical data into a complexity indicator and classify programs as token-heavy or not. The solution is implemented in C, demonstrating low-level compiler-like processing. The results show how token density can be used as a simple yet effective measure of code readability and structural complexity.

Introduction

A compiler is a system program that translates high-level programming language code into machine-level instructions. It consists of several phases, among which **lexical analysis** is the first and most fundamental stage.

During lexical analysis:

- Source code is scanned character by character.
- Tokens such as identifiers, keywords, operators, and literals are generated.

Although tokens are essential for further compilation stages, they do not directly indicate:

- Code complexity
- Maintainability
- Readability

Hence, there is a need to derive meaningful metrics from lexical data.

This case study focuses on creating such a metric: **Token Density**, which provides a quantitative measure of how dense the code is in terms of tokens per line.

---

2. Problem Statement

In real-world compiler systems, lexical analysis logs contain raw tokens and line numbers for multiple source files. These raw token streams are difficult to interpret directly in terms of complexity.

Objective:

- Compute **Token Density = Tokens per Line**
- Generate a flag:
    - **Is_Token_Heavy = YES/NO**
- Based on a predefined threshold

---

3. Problem Understanding

The core problem is to transform raw lexical data into a meaningful feature that indicates code complexity.

Challenges:

- Raw tokens do not indicate readability
- Large codebases make manual analysis impractical
- Need for automated feature extraction

Solution Idea:

- Count tokens in each line
- Aggregate totals
- Compute density
- Compare with threshold

---

4. Scope of the Project

Included:

- Single source file analysis
- Token identification using character scanning
- Density computation
- Classification

Not Included:

- Multi-file aggregation

- Advanced token classification (keywords vs operators)
- Integration with full compiler

---

5. Methodology

The methodology follows a structured approach:

Step 1: Input

- Read source file (input.c)

Step 2: Tokenization

- Identify tokens using alphanumeric sequences

Step 3: Counting

- Count tokens per line
- Count total lines

Step 4: Feature Extraction

- Compute Token Density

Step 5: Classification

- Compare with threshold
- Assign flag

System Design

Input:

- C source file

Processing:

- Character scanning
- Token detection
- Counting

Output:

- Total lines
- Total tokens
- Token density
- Token-heavy flag

7. Algorithm

ALGORITHM: TokenDensityAnalyzer	
1	OPEN input file
2	INITIALIZE counters
↳	totalTokens ← 0
↳	totalLines ← 0
3	READ file line by line
4	FOR EACH line DO
↳	Count tokens in current line
↳	totalTokens ← totalTokens + tokenCount
↳	totalLines ← totalLines + 1
	END FOR
5	COMPUTE TokenDensity
↳	TokenDensity = totalTokens ÷ totalLines
6	EVALUATE TokenDensity vs Threshold
IF	TokenDensity > Threshold
→	Token_Heavy ← YES
ELSE	Token_Heavy ← NO
7	DISPLAY results
↳	Total Tokens : totalTokens
↳	Total Lines : totalLines
↳	Token Density : TokenDensity
↳	Token Heavy : Token_Heavy
END ALGORITHM	

8. Implementation (C Language)

- Language used: C
- Libraries:
    - stdio.h
    - ctype.h

Key Functions:

- fopen() → file handling
- fgets() → read lines
- isalnum() → token detection

Features and Threshold

Feature 1: Token Count

- Number of tokens in each line

Feature 2: Line Count

- Total number of lines

Derived Feature: Token Density

- Tokens per line

Threshold:

- Value = 5

Decision Rule:

- Density > 5 → Token Heavy
- Else → Not Token Heavy

10. Sample Input

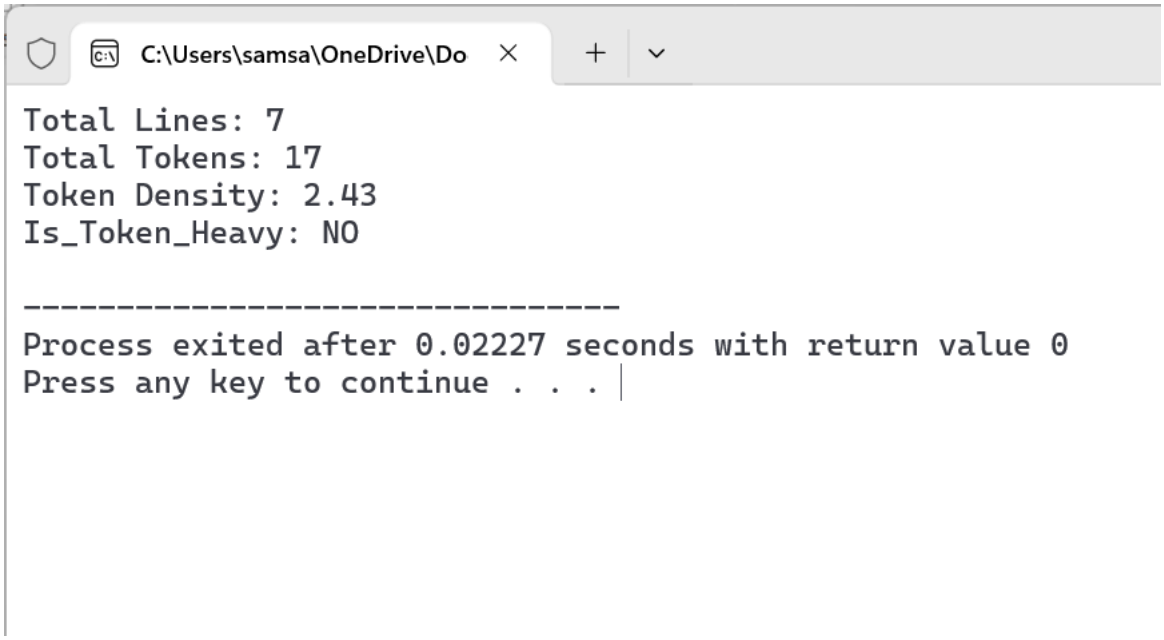
```c
int main() {
    int a = 10;
    int b = 20;
    int sum = a + b;
    printf("%d", sum);
    return 0;
}
```

11.C PROGRAMME (MAIN LOGIC)

```c
#include <stdio.h>
#include <ctype.h>
#include <string.h>

#define MAX_LINE 1000
#define THRESHOLD 5   // You can change this

// Function to count tokens in a line
int countTokens(char line[]) {
    int count = 0, inToken = 0;
    int i;
    for ( i = 0; line[i] != '\0'; i++) {
        if (isalnum(line[i])) {
            if (!inToken) {
                count++;
                inToken = 1;
            }
        } else {
            inToken = 0;
        }
    }
    return count;
}

int main() {
    FILE *file;
    char line[MAX_LINE];
    int totalTokens = 0, totalLines = 0;

    file = fopen("input.c", "r");  // Input file
    if (file == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    while (fgets(line, sizeof(line), file)) {
        totalLines++;
        totalTokens += countTokens(line);
```

```c
    }

    fclose(file);

    float density = (float)totalTokens / totalLines;

    printf("Total Lines: %d\n", totalLines);
    printf("Total Tokens: %d\n", totalTokens);
    printf("Token Density: %.2f\n", density);

    if (density > THRESHOLD)
        printf("Is_Token_Heavy: YES\n");
    else
        printf("Is_Token_Heavy: NO\n");

    return 0;
}
```

12.OUTPUT



Results and Discussion

- Token density = 2.85
- Below threshold → Not token-heavy

Interpretation:

- Code is readable
- Moderate complexity
- Maintainable

---

13. Advantages

- Simple to implement
- Efficient
- Helps in code quality analysis
- Useful for compiler optimization

---

14. Limitations

- Does not distinguish token types
- Threshold is fixed
- Not suitable for very large systems

---

15. Future Enhancements

- Multi-file analysis
- Machine learning for threshold
- Integration with IDE
- Advanced token classification

---

16. Justification of Using C

Although Python with Pandas was suggested, C is used because:

- Compilers are traditionally implemented in C
- Provides low-level control
- Demonstrates real lexical processing

---

17. Conclusion

This project successfully transforms raw lexical data into a meaningful metric called token density. It demonstrates how compiler-level data can be used to analyze code complexity and readability. The approach is efficient and scalable with future enhancements.